

React hooks cheatsheet

State

● `useState()`

Declares a state variable.

```
const [age, setAge] = useState(42);
```

● `useReducer()`

Declares a state variable managed with a reducer.

```
const [state, dispatch] = useReducer(reducer, { age: 42 });
```

Context

● `useContext()`

Reads and subscribes to a context.

```
const theme = useContext(ThemeContext);
```

React hooks cheatsheet

Effects

● `useEffect()`

Synchronize external state.

```
useEffect(() => {  
  const unsubscribe = socket.connect(props.userId);  
  
  return () => {  
    unsubscribe();  
  }  
}, [props.userId]);
```

● `useLayoutEffect()`

Read layout DOM state.

```
useLayoutEffect(() => {  
  // Read DOM layout  
});
```

React hooks cheatsheet

Effects

● `useInsertionEffect()`

Insert styles into the DOM.

```
useInsertionEffect(() => {  
  // Insert styles  
});
```

Subscribing

● `useSyncExternalStore()`

Subscribe to external state.

```
const state = useSyncExternalStore(subscribe, getSnapshot);
```

React hooks cheatsheet

Memoization

● useCallback()

Return a memoized callback.

```
const handleClick = useCallback(() => {  
  doSomething(a, b);  
}, [a, b]);
```

● useMemo()

Return a memoized value.

```
const value = useMemo(() => {  
  return calculateValue(a, b);  
}, [a, b]);
```

React hooks cheatsheet

Refs

- **useRef()**

Declares a ref.

```
const inputRef = useRef(null);
```

- **useImperativeHandle()**

Customize instance value exposed to parent refs.

```
useImperativeHandle(ref, () => {  
  // ...  
});
```

React hooks cheatsheet

Accessibility

● `useId()`

Generate unique IDs across the server and client.

```
const id = useId();
```

Debugging

● `useDebugValue()`

Display a label for custom hooks.

```
useDebugValue('Custom Label');
```

React hooks cheatsheet

Events

● `useEvents()`

Extract non-reactive effect logic into an event function.

```
useEvent(callback)
```

Transitions

● `useDeferredValue()`

Defer to more urgent updates.

```
const deferredValue = useDeferredValue(value);
```

● `useTransition()`

Lets you mark updates in the provided callback as transitions.

```
startTransition(() => {  
  setCount(count + 1);  
})
```